Singularity Health Center: Execution And Architecture Narrative

Executive Summary

Singularity Health Center should be delivered as a real-time operational intelligence platform for endpoint health, not merely as another console dashboard. The business problem is customer trust: customers need to know whether their SentinelOne agents are healthy, protected, connected, and actionable across millions of endpoints. The engineering problem is to process billions of daily health events, detect high-impact anomalies within five minutes, and serve a responsive user experience under 200 ms p95 API latency. The leadership problem is to deliver this while the team is still responsible for a legacy heartbeat service that is already causing customer escalations.

My approach is to split the initiative into a focused MVP, a GA hardening phase, and a platformization phase. The MVP should cover the highest-value health signals, basic alert coalescing, a tenant-scoped dashboard, operational SLOs, and a safe tenant allowlist rollout. GA should expand lifecycle management, replay/backfill, richer search and drill-down, and validated migration away from legacy offline logic. Platformization should turn the rule evaluation, coalescing, alert lifecycle, and health telemetry contracts into reusable Agent Platform capabilities.

The architecture should use a durable event bus, stream processing, an operational alert store, a read-optimized search model, and a long-term data lake. This cloud-neutral design can map to Kafka, Pub/Sub, or Kinesis for event transport; Flink or Kafka Streams for processing; Cassandra, DynamoDB, or Bigtable for high-write alert state; OpenSearch or Elasticsearch for dashboard queries; and S3/GCS-style object storage for historical retention and replay.

Requirements And Product Scope

The core functional requirements are telemetry ingestion, anomaly detection, alert coalescing, alert lifecycle, dashboard APIs, and UI workflows. For MVP, I would limit anomaly types to agent offline/connectivity loss, agent disabled, anti-tamper disabled, and low disk or local resource risk. These are high-signal operational failures that customers can understand and act on. I would avoid starting with a fully generic rule engine, advanced analytics, or broad notification workflow. Those are valuable, but they increase delivery risk before the platform has proven its basic correctness and reliability.

The non-functional requirements drive the architecture. Processing billions of events per day means the design cannot rely on synchronous service-to-service fanout or raw database writes followed by expensive queries. Detecting anomalies within five minutes requires near-real-time processing and explicit freshness monitoring. Serving dashboard APIs under 200 ms requires a read model shaped for the console experience rather than ad hoc queries over raw telemetry. Operating in a security product also implies tenant isolation, auditability, replay, idempotency, and clear blast-radius controls.

I would set the initial SLOs around event freshness, dashboard latency, and silent failure detection. For example: 99 percent of accepted health events should be processed

within five minutes; standard dashboard APIs should remain under 200 ms p95; and no pipeline stage should be able to silently stop processing for more than five minutes without paging. These SLOs are important because they turn broad reliability expectations into measurable engineering decisions.

Architecture Strategy

The proposed architecture separates ingestion, processing, serving, and analytics concerns.

Agents emit health telemetry through the central Ingestion Gateway. The gateway validates, authenticates, rate-limits, and routes events into a durable event bus. The Health Center team consumes those events through a versioned telemetry contract. This contract is critical: it gives the team an adapter boundary that protects downstream development from changes in the gateway implementation.

The event bus should be durable, partitioned, and replayable. Partitioning by tenant and agent ID helps preserve ordering where it matters and keeps coalescing state local to stream processors. At-least-once delivery is acceptable if processing is idempotent. Exactly-once semantics are attractive in design discussions, but I would not make them a dependency for MVP correctness. The safer model is to design alert state updates using deterministic keys, event IDs, rule versions, and idempotency tokens.

The stream processing layer performs real-time anomaly detection and alert coalescing. A typical example is grouping repeated "anti-tamper disabled" events from the same agent over a ten-minute window into one alert. The processor should emit state transitions, not raw event spam: open alert, update evidence count, resolve alert, suppress duplicate, or escalate severity. This keeps downstream stores and user workflows focused on actionable health states.

For storage, I would use three logical stores. First, an operational alert store optimized for high write throughput and tenant/agent lookups. Second, a search or read model optimized for dashboard filters, aggregations, and alert lists. Third, a data lake for long-term telemetry retention, audit, replay, and rule tuning. This avoids forcing one database to serve all access patterns.

The API layer should read from precomputed views. Dashboard summaries, alert counts, top affected groups, and recent alert lists should not scan raw telemetry. Query shapes should be bounded, paginated, and tenant-scoped. Where freshness matters, APIs should expose freshness metadata so the console can distinguish "no issues" from "data may be delayed." This is especially important in a security operations product, where silent staleness is worse than an explicit degraded state.

Several components should be designed as reusable platform capabilities: telemetry schema governance, rule evaluation, alert coalescing, alert lifecycle, tenant rollout, replay/backfill, and pipeline health monitoring. I would not over-generalize them before MVP, but I would keep boundaries clean enough that future Agent Platform use cases can adopt them.

Alert Coalescing Trade-Off

The alert coalescing disagreement between Engineer A and Engineer B is a useful

leadership moment. Engineer A's stream-processing proposal better fits the freshness and scale requirements, while Engineer B's database-and-cron approach has the virtue of simpler initial infrastructure. Both positions are valid because they optimize for different risks.

I would mediate by moving the discussion from preference to decision criteria: detection latency, write amplification, query cost, operational complexity, replay behavior, failure isolation, and delivery timeline. Then I would ask the engineers to run a short design spike with realistic cardinality, duplicate-event rates, and failure cases. The output should be a decision record, a rough load model, and a rollback plan.

My expected decision is to use stream processing for MVP coalescing, but with constrained scope. We should not build a broad generic rules platform first. We should implement a small number of explicit rules, a reusable windowing/coalescing abstraction, and strong instrumentation. The reason is simple: the requirement says anomalies must be detected within five minutes at very high event volume. Database-first cron aggregation risks late detection, expensive scans, and noisy intermediate state. Stream processing adds complexity, but it puts that complexity in the part of the system designed for time-windowed aggregation.

Once the decision is made, I would ask both engineers to help own the outcome. Engineer A can lead stream-processing design and operational readiness. Engineer B can lead simplicity controls: bounded rule scope, failure-mode review, cost model, and clear runbooks. That turns disagreement into a stronger design instead of a winner-loser moment.

Operational Excellence And Sev-1 Handling

The most dangerous failure mode is not a visible outage. It is Health Center silently stopping event processing while the UI continues to serve stale or incomplete data. To prevent that, every pipeline stage needs freshness and volume monitors: gateway ingress rate, event-bus lag, processor lag, dropped-event count, alert-store write rate, read-model indexing lag, API latency, and synthetic event success.

I would add synthetic health events that flow end to end through the same path as production telemetry. These synthetic events should be scoped by region and tenant cohort, and they should page if they do not appear in the alert/read model within the freshness SLO. I would also create dashboards that show the pipeline by stage, not only aggregate service health. A Kubernetes pod being "up" is not proof that the platform is processing events correctly.

For a Sev-1 where processing silently stops, the response should be disciplined. First, declare the incident and assign clear roles: incident commander, communications lead, and technical leads for ingestion, processing, storage, and API. Second, determine blast radius by region, tenant cohort, event type, and time window. Third, isolate the failed stage by comparing gateway accepted events, event-bus offsets, processor checkpoints, alert-store writes, read-model updates, and synthetic probes. Fourth, mitigate by restarting or failing over processors, disabling a bad rule, pausing rollout, or routing to a fallback consumer. Fifth, replay the durable event log from the last known good offset and backfill missing alert transitions. Finally, publish a blameless postmortem with corrective actions such as stronger synthetic coverage, improved circuit breakers, and better deployment gates.

Execution Roadmap

In Quarter 1, I would focus on foundations and MVP. The team should finalize telemetry contracts with the Ingestion Gateway team, create the ingestion adapter, implement the stream processing skeleton, define alert state schemas, build basic APIs, and ship the first dashboard behind feature flags. The anomaly catalog should stay narrow: offline/connectivity loss, agent disabled, anti-tamper disabled, and low disk/resource risk. Operational work is part of MVP, not a later hardening activity: dashboards, alerts, runbooks, load tests, and synthetic events must exist before customer rollout.

In Quarter 2, the team should move toward GA readiness. This includes scale testing, replay and backfill tooling, alert lifecycle, suppression, richer search filters, tenant expansion, and operational game days. This is also where I would run shadow comparison between Health Center offline status and the legacy heartbeat service. We should not flip source-of-truth logic until we can quantify false positives, false negatives, and customer impact.

In Quarter 3, the focus becomes platformization and migration. The team can migrate selected offline decisions from the legacy service to Health Center, expand the anomaly catalog, add historical trends and reporting, and expose reusable platform APIs. This phase should also reduce operational burden by retiring duplicated legacy logic where safe.

Dependency Delay Plan

If the Ingestion Gateway team is delayed by two months, I would not allow the Health Center team to become idle. The first move is to freeze a versioned telemetry contract and build a Health Center adapter against that contract. Then the team can develop against synthetic streams, replayed historical events, or existing heartbeat and agent metadata feeds. This unblocks stream processors, storage schemas, APIs, UI, QA automation, observability, and load testing.

I would also negotiate a thin-slice integration with the gateway team. Instead of waiting for every health event type and full routing capability, we can ask for minimal routing of one or two high-value signals first. That lets us validate the production integration path while keeping the full contract on a later milestone. The roadmap impact should be explicit: dependency-bound scope moves, but downstream platform construction and UI delivery continue.

Operational Drain From Legacy Offline Escalations

The 40 percent spike in false "Offline" escalations must be treated as customer trust risk. Ignoring it would damage the credibility of the new Health Center before it launches. At the same time, moving the entire team to the legacy issue would jeopardize the strategic platform.

I would create a short-lived stabilization lane: two engineers and targeted QA support focus on the legacy heartbeat issue for two to three weeks. Their mission is narrow: instrument the failure, identify the top false-offline causes, patch the highest-impact issue, and reduce support volume. Six engineers remain focused on the Health Center MVP, while one tech lead coordinates shared interfaces and migration implications. The SEM

owns prioritization, stakeholder communication, and explicit escalation criteria.

This plan protects both customer trust and roadmap momentum. It also turns the legacy issue into useful input for Health Center. Every false-offline root cause should become a test case, rule refinement, or migration guardrail in the new platform.

People Leadership And Morale

Morale will not be solved by superficial recognition if engineers feel trapped between roadmap pressure, incidents, and unclear priorities. I would keep the team engaged by making priorities explicit, protecting focus time, and giving engineers ownership of meaningful areas.

The operating model should include clear workstreams, a visible decision log, weekly risk review, and explicit WIP limits. Engineers working on stabilization should know what "done" means so they are not permanently trapped in interrupt work. Engineers working on Health Center should see customer impact early through metrics, dogfooding, tenant feedback, and reduced support escalations.

I would create growth opportunities aligned with the project: one engineer leads stream processing design, another owns API/read-path performance, another owns UI workflows, another owns replay/backfill, and another owns observability and incident readiness. Senior engineers should present design decisions in architecture review, not only implement tickets. QA should be involved in failure-mode design and rollout gates from the beginning, not treated as a late-cycle validation function.

The leadership message to the team is: we are solving a real customer pain, building a platform that outlives the first release, and doing it in a way that does not burn people out. That requires focus, candor, and disciplined trade-offs.

Closing

The Health Center program succeeds if it improves customer trust while creating a durable Agent Platform foundation. The architecture must handle hyperscale telemetry, but the bigger test is execution judgment: narrow MVP scope, protect customer-facing reliability, absorb dependency delays, resolve technical conflict with evidence, and keep the team engaged through operational pressure. This is the operating posture I would bring as the Senior Engineering Manager for Agent Platform.